

National College of Ireland

Project Submission Sheet

Student Name: Uday Kiran Reddy Dodda

Student ID: 25166484

Programme: MSC In Cloud Computing **Year:** 2026-2027

Module: Research In Computing

Lecturer: Dr. Sean Heeney

Submission Due Date: August 07 , 2026

Project Title: Evaluating MQTT Quality-of-Service Reliability in AWS IoT Core for Device-to-Cloud Telemetry under Intermittent Connectivity

Word Count:

Xb0v9n5MjI

Academic Integrity Certificate Verification Code:

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting Monday, 19th January 2026, all students are required to successfully complete the Epigeum Academic Integrity Course, which is available on Moodle. This is a self-paced online course consisting of 4 modules and will take approximately 2 hours to complete. Once you have successfully completed the course, you will receive a Certificate & Verification Code. This code must be included on the Project Cover Sheet when submitting your work. For full details and guidance, please visit: <https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>

Signature: Uday Kiran Reddy Dodda

Date: 07 August 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplements in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

Research In Computing

Evaluating MQTT Quality-of-Service Reliability in AWS IoT Core for Device-to-Cloud Telemetry under Intermittent Connectivity

Your Name/Student Number	Course	Date
Uday Kiran Reddy Dodda	MSC In Cloud Computing	2026-2027

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click [here](#).

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used.**

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

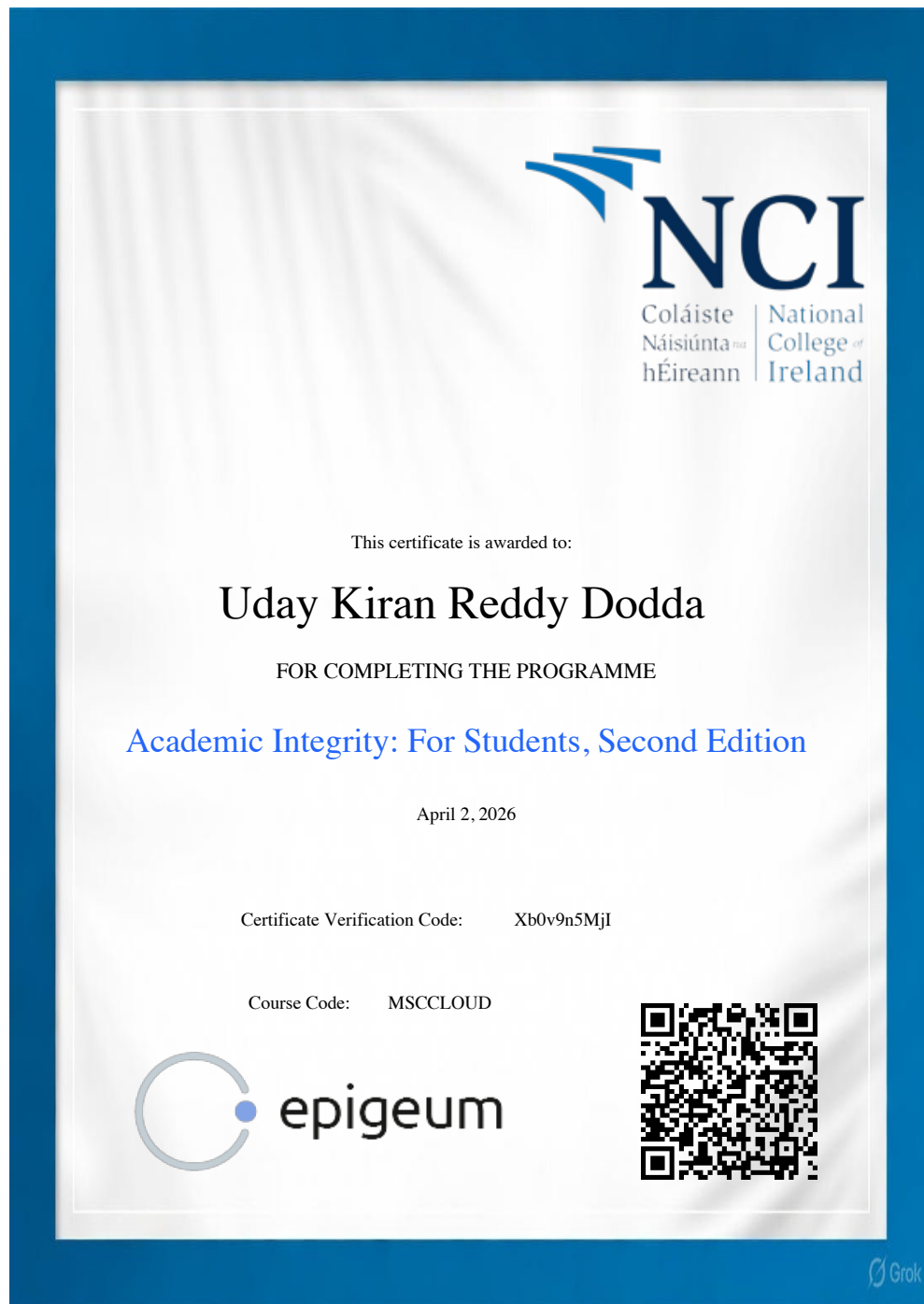
[Place evidence here]

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field

Academic Integrity Subject Guide or how to access the site: <https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>

Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI



Evaluating MQTT Quality-of-Service Reliability in AWS IoT Core for Device-to-Cloud Telemetry under Intermittent Connectivity

Uday Kiran Reddy Dodda

X25166484

MSc in Cloud Computing

National College of Ireland

Abstract

Over 80% of the observed MQTT traffic does not come with guarantees, which is reasonable on a stable network and irresponsible when devices go offline. For managed brokers, which applies is not measured because published comparisons are run on self-hosted brokers whose setup the experimenter controls, and one large-scale managed-broker study leaves unmeasured managed-broker quality-of-service and intermittent connectivity as future work. The proposal outlines a controlled disconnection test on AWS IoT Core that involves the simulation of a set of devices publishing the same telemetry at each service level, and a set of periodic disconnections of a fixed duration. All published messages are compared to their delivered record, therefore it is not assumed that they are lost or duplicated because that is counted; latency, reconnection time and service cost is reported along with them. What you get is a price to be surmised based on a guarantee that you take on faith.

Keywords: MQTT, quality of service, AWS IoT Core, message reliability, intermittent connectivity

1. Introduction

The data from connected devices is published to and subscribed from the cloud through a publish-subscribe protocol, and that protocol is nearly always MQTT. The publisher selects a quality of service (QoS) level for each message, and that's the only reliability control it has. Level 0 provides at least one level at most, and will not repeat. At least once that waits for the broker to acknowledge at the cost of duplication, is the contract level of delivery for level 1.

It is decided at design time and little changed throughout the running of the program. Innamorati et al. (2026) scanned the IPv4 address space and discovered 14,386 reachable brokers and roughly 3.2 billion messages were published over a span of two weeks, over 88% of which were at level 0. In deployed practice, no matter the protocol allows, the guarantee is almost always foregone.

It is not wise to have that default if that is the network you are using, and wireless devices do not have stable networks. Out of range, low batteries, then back in range minutes later. The protocol describes what

each level guarantees, and it doesn't describe what a broker does during such an interruption because this is part of its implementation—how long it keeps sessions, how deep is its queue, which is the expiration time for held message, etc., and whether a reconnecting client is throttled.

For a self-hosted broker those properties can be viewed and modified. On a managed service they can't. Here, AWS IoT Core resolves them by policy and exposes only levels 0 and 1 for publishing from devices to the cloud (Amazon Web Services, 2026). Once the tenant selects a level, he or she receives the rest and the impact of that decision is ultimately an empirical one, not a configuration one.

The consequence is thus an empirical one: How much less loss of telemetry messages will there be in AWS IoT Core when the publishing device is experiencing controlled disconnections for varying lengths of time, publishing at level 1 instead of level 0?

The evidence available is bi-directional. Ko and Lee (2025) found that level 0 did not lose as many of the messages (2.10%) over the stable laboratory network, at a mean latency of 95.0ms compared to level 1 at 150.2ms, which makes the common default of level 1 seem like a decent compromise. In fact, default settings for reconnecting publishers lose around half of all messages, as measured by Lima et al. (2025), with no messages lost at level 1; this seems indefensible. They both were measured on brokers that the authors operated.

Neither reading has any impact on managed endpoint, as the machine that decides the reading is the provider's. The platform that the protocol governs is not guaranteed, the protocol does not guarantee what it delivers across an interruption.

This can be split into four objectives. The first one measures the loss of messages at each service level during the controlled-duration disconnections. The second measures duplication and end-to-end latency, the cost of any reliability obtained. The third measures the reconnection behaviour and the amount of the backlog that remains after reconnection. The fourth shows each configuration as a point on the resulting reliability-cost surface.

The proposed solution is a controlled disconnection experiment and not an observational study. Each message has an identifier that is stored in a device-side log before it is published and each delivered message is correlated against this stored identifier, not based on an assumption that the message should have been delivered, based on a broker-side counter.

The contribution is a fee paid for a guaranteed number of telemetry messages remaining after a disconnection at each level on a managed broker, the duplication and latency that these messages incur, and the cost of those duplicates and latency to the bill. Practitioners acquire evidence for a decision that is made in code, and used infrequently.

Section 2 summarizes the findings of deployed practice and stable-network measurement and what the impairment studies have found about the behaviour of the broker under stress, and demonstrates why neither is sufficient to answer the question for a managed endpoint. The experimental design, the metrics and statistical treatment, the ethical position and the schedule are determined in Section 3.

2. Literature Review

A. *Quality of Service in Deployed MQTT Practice*

There are two studies that provide a range of answers to the question of stable networks and different answers about the default costs. The default is set by Innamorati et al. (2026) who find over 88% of the 3.2 billion messages captured with level 0 across 14,386 brokers. Ko and Lee (2025) then report the cost of that in a working deployment, 2.10% packet loss, 95.0 ms latency for level 0 and 99.2% delivery, 150.2 ms for level 1. Together, they indicate a sensible trade: a small loss rate purchased for a third off the latency.

The trade is only valid while the link is active. Jeddou et al. (2022) illustrate the risks of transferring such figures, and conclude that the ordering of the services by delay changes as the link round-trip time increases: a finding that an ordering determined on a local network does not necessarily reflect that on a wide-area path to a regional cloud end-point. A lab measurement then sets a trade for one network condition and then can't be extrapolated to another.

What is deployed practice and stable network measurement establishing, then, is the default and its cost, when nothing goes wrong? What happens when something does is considered next.

B. *Measuring Broker Reliability under Impairment*

The picture is changing rapidly in impairment studies. Gruener et al. (2021) killed a broker mid-run to study how that affects the ability of the underlying network to maintain itself as a level 0 delivery network. With 30% packet loss, the ability to deliver level 0 messages dropped to 83% for Gruener et al.'s implementation while almost no messages were lost and duplicated for other implementations. Gruener et al. also experimented by degrading the network below several brokers and found that with 30% packet loss, level 0 delivery dropped to 83%, and level 1 worst-case tail latencies were thirty seconds or longer for their implementation while the other implementations had latencies of less than 10 seconds. Lima et al (2025) then focused on publisher churn alone: around 50% churn at level 0, and 0% churn at level 1.

The two results are the closest to this one and both are intimately connected and have the decisive disadvantage: Both are instrument brokers that the authors operate. Gruener et al. damage the network under a broker that they control, while Lima et al. experiment with bridge architectures that they setup. Both monitor the disconnection of a managed endpoint without having any influence over its session retention, queue depth, message expiry, and reconnection throttling, which are controlled by the provider.

This study builds on the work of Shvaika et al. (2025) which shows a Kafka- and Redis-backed broker keeping messaging running for more than one hundred million clients and characterizes its through and persistence under a steady high load. They clearly say that the different service levels, and the possibility of intermittent connectivity and different situations of the client "are left as future work. This is not an aesthetic issue: A lack of convergence between the brokers Gruener et al. (2021) uncover is not reflected in the throughput measures they considered in steady-state; it is a scaling ability and survival from interruption issue.

Their subsequent study serves as a reinforcement of the same point but from the other side. For persistent-session subscribers, Shvaika, Shvaika and Artemchuk (2025) isolate the offline-message store, finding it saturated between 30,000 and 60,000 messages per second when all clients are connected. It can then be described as the capacity path of the replay, which is never used during a real reconnection, that is when it is needed.

Dizdarević et al. (2024) benchmark several broker implementations over a continuum of compute, and close but do not merge the two conditions, but rather assume that the connection is stable and set their impairment-sensitive tests at level 0. This causes their degraded-network and loss-sensitive conditions never to coincide and no client ever disconnects. Panagou et al. (2025), in their review of the managed IoT platforms which included this platform, acknowledge that their study is based on the documentation from the vendors and free-tier trials with high stress load testing that was out of scope.

The reviewed studies are compared to this proposal in Table 1. All of the columns that are correlated are handled by the broker (these are the ones that matter), no reviewed row has all three.

Table 1 Reviewed studies contrasted with the proposed experiment

Study	Broker	Service levels varied	Client disconnection	Per-message ground truth	What it leaves open
Innamorati et al. (2026)	Observed, in the wild	Observed only	No	No	Whether the observed default is rational under loss
Ko and Lee (2025)	Self-hosted	Yes	No	Partial	Behaviour once the link is impaired
Gruener et al. (2021)	Self-hosted	Yes	Network impaired, not client	Yes	Managed-broker behaviour
Lima et al. (2025)	Self-hosted bridge	Yes	Publisher churn	Yes	Provider-governed session retention
Dizdarevic et al. (2024)	Managed and self-hosted	Fixed at level 0 under impairment	No	No	Level comparison under impairment
Shvaika et al. (2025)	Self-hosted, purpose-built	No, steady load	No	No	Named as future work by the authors
This proposal	Managed (AWS IoT Core)	Yes, both available levels	Yes, controlled duration	Yes, device-side log	Answers the above for a managed endpoint

C. *Why This Research Is Necessary*

There are three things that have been decided. The default has been determined: Level 0 is the dominant traffic in the deployed traffic (Innamorati et al., 2026). The default cost on a stable network is established at a couple of percent of messages for about a third of latency (Ko and Lee, 2025). And its behaviour in the event of broker-side impairment is resolved, and after a broker's collapse, there are differences in implementations (Gruener et al., 2021).

The question that remains is what happens when a broker that is not the tenant's is disconnected on their client side. All impairments above are measured at a fixed service level, as done by Dizdarević et al. (2024); the gap is identified as future work by the baseline; and the associated measure is taken only during periods when clients are connected. The single review for the managed platform addressed is based upon documentation, not measurement.

The practical effect is that if a fleet is experiencing a high percentage of downtime, then there is no way for a team to measure the fleet to determine what service level is appropriate, nor can they do so from the specification as that is what the provider does to it. This proposal provides this measurement.

D. *Research Niche*

No existing study has been performed to measure message loss along a client-side disconnection, at more than one service level, on a managed broker, and comparing the message loss to per-message ground truth.

None of these four conditions are met at once in the studies we reviewed: Gruener et al. (2021) vary the level and impairment, but implement it on their own broker, which they host themselves; Lima et al. (2025) disconnect publishers but instrument their own bridge; Dizdarević et al. (2024) measure managed and self-hosted deployments, but leave the level fixed under impairment. Every reviewed design is able to answer its own question well, and no one had built one to answer this.

The baseline is from Shvaika et al. (2025). Their design considers a broker they construct, under the condition that it's the least important one, and it states the unexamined conditions. It is that absence that is addressed here. This study turns the tables, measuring a broker the tenant cannot change when the conditions in the baseline are non-differential, and keeping delivery completeness as the common measure, so that results can be compared against the baseline.

To allow a measure to challenge the two expectations are documented as thresholds: Threshold 1: The number of dropped messages is less than 1/10 of that at the Threshold 0 with disconnection times ≥ 1 minute; Threshold 2: The duplication rate is $> 1\%$ (the cost side of the same guarantee). A third one is left open as there is no evidence reviewed to predict if held messages will remain for a long disconnection on this platform.

Experiment is described in Section 3. These are service level 2, which this platform does not support for publish to cloud from devices; multi-region ingestion; physical devices not simulated; and other brokers besides the one measured here.

3. Research Method & Specification

A controlled disconnection experiment answers this question. Simulated devices publish identical telemetry at each service level while disconnections of fixed duration are imposed on a schedule, and every message is matched against a device-side record. Figure 1 shows the arrangement.

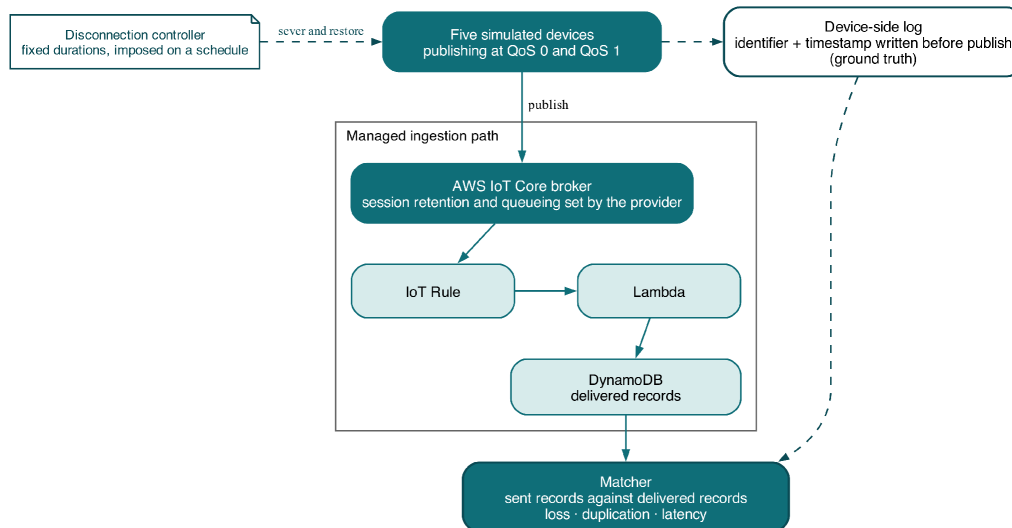


Figure 1 Experimental setup: publication path, disconnection control and matching

A. Research Method

There are two between-subjects factors. The service level is either 0 or 1, with Amazon Web Services only exposing these for the device-to-cloud publish part of the service; therefore, the study is not artificially limited, but leverages the service under study. The disconnection duration is divided into four levels: none, 15 seconds, one minute and 5 minutes, representing the range in which a held message might plausibly persist, die or go out of memory. In addition to the factors mentioned above, the publication rate is also a conditioner, at two levels: steady, bursty, because the backlog that gets built up during interruption is what the reconnection has to clear.

The dependent variable is the number of published messages lost end to end out of a total number of published messages. Secondary measures include the duplicate-delivery rate, end-to-end latency at the mean, end-to-end latency at the 95th percentile of the latency distribution and at the 99th percentile, reconnection time, rule-trigger success rate and estimated service cost per configuration. A team of 12 high school students from the Perth High School is delivering the results on the same terms as Shvaika et al. (2025) so that the result for the managed-broker can be read against their steady-load account.

Loss is only measurable when there is ground truth. The population of intended messages is known independently of the broker since each simulated device writes a message identifier and a timestamp to a local log just prior to publishing. Delivered records are compared to that log on identifier; if it is not found downstream, it is considered to be a loss, and if it is found twice, it is considered to be a duplicate. If there were no device-side log, it would be possible to only infer loss from broker counters, which is what Innamorati et al. (2026) is limited to observing.

The region, the account, the device count of 5, the size of the payload and the client library and its version are all assumed to be constant. There are two confounds that need to be addressed. Disconnections are not expected, but need to be forced, or their length is not known; they disconnect from the client and reconnect on a timer. A reconnecting client can be throttled separately like loss and the reconnection time is measured directly and reported next to the loss.

The devices are virtual processes and not actual hardware, therefore, radio behavior and power management are not included and the disconnection is more "clean" than a real one. The external validity is limited to one managed platform and one region. In construct terms, estimated cost is calculated based on unit price cost published from counted operations instead of the bill and the derivation is written in such a way that it can be computed again.

B. Research Resources

The experiment uses AWS IoT Core as the broker, a set of rules to route the delivery of each message to a Lambda function, and a DynamoDB table that stores the delivery records to which the device-side log is compared. All are provisioned from infrastructure code which ensures that each configuration is identically created and destroyed, and spends remains bounded.

The test data is synthetic telemetry data created for this study consisting of fixed-size numeric sensor readouts and a sequence identifier. No personal data, no production traffic and no 3rd party data is ever used, and there is no participation. For each configuration, the 5 simulated devices publish 1000 messages at 5-second intervals, and the total campaign does not go into the "free-tier unsafe" range, but is still sufficiently large for a reasonable estimate of the loss rate, from a few percent up.

The device simulator, the infrastructure definitions and the measurements gathered are released and therefore any figure reported here can be recomputed from source, not on trust.

C. Evaluation

The comparison of the loss rates at each disconnection duration between the two service levels, which addresses Objectives 1 and 2, settled the question. Objective three is derived from reconnection-time and backlog measurements made during reconnection, while objective four is derived by plotting each configuration on the surface spanned by loss, duplication and cost.

A comparison is written in terms of a null and an alternative hypothesis prior to the collection of any data. H_0 believes that there is no difference between the two service levels with a disconnection time, in terms of the proportion of messages lost. H_1 believes that there is a difference between the two service levels with a disconnection time, in terms of the proportion of messages lost. Duplication: H_0 : no difference between levels in duplication rate; H_1 : that there is a difference. On latency, H_0 says that there is no difference in the mean end-to-end latency, while H_1 says that there is. Losses and duplications are directional: the protocol's definition dictates the direction, and the question is just how many; latencies are not.

Loss and duplication are proportions of a known population and each level pair is compared via a two-proportion z-test and the interaction between service level and disconnection duration is analyzed across the entire contingency table by a chi-squared test. The data are very right skewed on the tail, so normality is first tested and, if not, a Mann-Whitney U test with rank-biserial correlation is used. All reported differences are 95% confidence intervals.

Test is performed at $\alpha=0.05$ and Holm-Bonferroni adjustment because each of the hypotheses refers to one measurement campaign. Replication is predetermined: Each of the sixteen configurations is replicated five times, for 5000 published messages per configuration, which provides sufficient resolution to distinguish 1% point change in the loss at 80% power. A difference that is considered practically significant is predetermined, at one per cent loss and five per cent cost.

D. Ethical Considerations of the Research

No one participates, and no personal data is ever made use of. All messages are synthetic telemetry produced for the experiment and all devices are synthetic processes, so there is no scope for consent nor data protection duties. The considerations that do come up are different.

If traffic is being sent to a commercial service, it should not be sustained or bursty traffic, and must be in compliance with provider acceptable use policies. All traffic is from researcher's own account, is directed towards resources created for this study, and is within free tier safe boundaries. Deliberate disconnection takes place at client level and never against the platform, so no pattern to the denial of service.

There is a cost to measurement: each message published and each rule invoked costs. The run count is not designed to be as high as the budget permits, but follows the power calculation in order to limit spending and thereby the energy (and hence the cost to the environment) in obtaining the result.

A special case of reporting integrity is that the study is for a named commercial service. Before there is any data, an analysis plan and cost derivation are agreed upon. If a message has been lost and/or duplicated, that's the report, instead of noise. Each statement of results comes with its own one-region, one-account scope of simulated devices. If the measurements show platform behavior that hasn't been documented, it will go to the provider before publication.

E. Project Plan

Figure 2 sets out the twelve-week schedule. Weeks one to three build and validate the rig: the device simulator, the ingestion path and the matching procedure, ending in a pilot that confirms a message published locally can be traced to a delivered record.

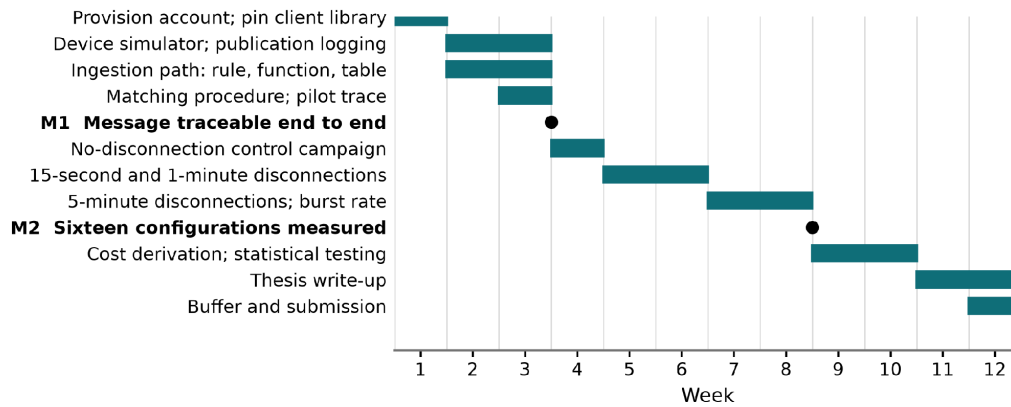


Figure 2 Twelve-week project plan

Measurement then proceeds by disconnection duration from week four, with the steady and bursty publication rates run against each. Weeks nine to eleven cover cost derivation and statistical testing, the write-up begins in week eleven, and week twelve stays free as contingency. The principal risk is that platform-side throttling proves hard to separate from loss, which the pilot is designed to expose.

4. References

- Amazon Web Services (2026) *AWS IoT Core developer guide: MQTT*. Available at: <https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html> (Accessed: 2 August 2026). [Vendor documentation - cited for the supported service levels only]
- Dizdarevic, J., Michalke, M., Jukan, A., Masip-Bruin, X. and D'Andria, F. (2024) 'Benchmarking performance of various MQTT broker implementations in a compute continuum', in *Proceedings of the 2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing*. Philadelphia: IEEE, pp. 357-366. doi: 10.1109/CCGrid59990.2024.00048. [Conference - CORE Ranking B (ICORE2026)]
- Gruener, S., Koziol, H. and Ruckert, J. (2021) 'Towards resilient IoT messaging: an experience report analyzing MQTT brokers', in *Proceedings of the 2021 IEEE 18th International Conference on Software Architecture*. Stuttgart: IEEE, pp. 69-79. doi: 10.1109/ICSA51549.2021.00015. [Conference - CORE Ranking A (ICORE2026)]
- Innamorati, C., Boiano, A., Redondi, A.E.C. and Cesana, M. (2026) 'The pulse of MQTT in the wild: a large-scale traffic analysis', *Computer Networks*, 274, 111845. doi: 10.1016/j.comnet.2025.111845. [Journal - JCR Impact Factor 4.7 (2025), Q2]
- Jeddou, S., Fernandez, F., Diez, L., Baina, A., Abdallah, N. and Agüero, R. (2022) 'Delay and energy consumption of MQTT over QUIC: an empirical characterization using commercial-off-the-shelf devices', *Sensors*, 22(10), 3694. doi: 10.3390/s22103694. [Journal - JCR Impact Factor 3.5 (2024), Q2]
- Ko, K.I. and Lee, M.H. (2025) 'MQTT-based architecture for real-time data collection and anomaly detection in smart livestock housing', *Sensors*, 25(23), 7186. doi: 10.3390/s25237186. [Journal - JCR Impact Factor 3.5 (2024), Q2]

- Lima, K., Oyetoyan, T.D., Heldal, R. and Hasselbring, W. (2025) 'Evaluation of MQTT bridge architectures in a cross-organizational context', in *Proceedings of the 2025 IEEE 22nd International Conference on Software Architecture*. Odense: IEEE, pp. 243-254. doi: 10.1109/ICSA65012.2025.00032. [Conference - CORE Ranking A (ICORE2026)]
- Panagou, I.C., Katsoulis, S., Nannos, E., Zantalis, F. and Koulouras, G. (2025) 'A comprehensive evaluation of *IoT* cloud platforms: a feature-driven comparative analysis', *IoT*, 6(2), 27. doi: 10.3390/iot6020027. [Journal - JCR Impact Factor 4.3 (2025), Q2; ESCI-indexed]
- Shvaika, A., Shvaika, D., Landiak, D. and Artemchuk, V. (2025) 'A distributed architecture for MQTT messaging: the case of TBMQ', *Journal of Big Data*, 12(1), 224. doi: 10.1186/s40537-025-01271-x. [Journal - JCR Impact Factor 10.8 (2025), Q1]
- Shvaika, D., Shvaika, A. and Artemchuk, V. (2025) 'MQTT broker architectural enhancements for high-performance P2P messaging: TBMQ scalability and reliability in distributed *IoT* systems', *IoT*, 6(3), 34. doi: 10.3390/iot6030034. [Journal - JCR Impact Factor 4.3 (2025), Q2; ESCI-indexed]